

Full Marks:50

SET:A

Time: 1Hour 10Minutes

Q1. What happens when this code runs?

```
def first():
    second()

def second():
    third()

def third():
    raise RuntimeError("Failure")
```

```
try:
    first()
except RuntimeError:
    print("Handled")
```

A)Handled
B)NameError
C)RuntimeError cannot propagate through functions
D)TypeError

Q2. What is the output?

```
try:
    x = int("abc")
except ValueError:
    print("A")
else:
    print("B")
finally:
    print("C")
```

A)A
B)B
C)A C
D)B C

Q3. What is the output?

```
def outer():
    x = 10

    def inner():
        return x + 5
```

return inner

```
f = outer()
print(f())
```

A)15
B)NameError
C)10
D)None

Q4. What does the following function return?

```
def fun():
    print("Hello")
```

```
x = fun()

print(x)
```

A)Hello
B>Error
C)Hello None
D)None

Q5. What will happen?

```
def fun(a, b=10):
    return a + b
```

```
print(fun(b=5, a=20))
```

A)TypeError
B)15
C)SyntaxError
D)25

Q6. What will be printed?

```
try:
    print(10 / 0)
except ZeroDivisionError:
    print("A")
except Exception:
    print("B")
```

A)Nothing
B)A



C)B

D)A B

Q7. What will happen?

```
def fun(n):  
    if n == 0:  
        return 1  
    return n * fun(n)
```

print(fun(3))

A)RecursionError

B)6

C)3

D)TypeError

Q8. What happens here?

```
def fun(a):  
    return a
```

print(fun())

A)NameError

B)None

C)0

D)TypeError

Q9. What will be printed?

```
class InvalidAgeError(Exception):  
    pass
```

```
def check_age(age):  
    if age < 18:  
        raise InvalidAgeError("Age is  
too low")  
    return "Allowed"
```

```
try:  
    print(check_age(15))  
except InvalidAgeError as e:  
    print(type(e).__name__)  
    print(e)
```

A)Exception

Age is too low

B)Nothing, because custom exceptions cannot be caught

C)InvalidAgeError

15

D)InvalidAgeError

Age is too low

Q10. What will be printed?

```
def make_counter():  
    count = 0
```

```
def counter():  
    nonlocal count  
    count += 1  
    return count
```

return counter

a = make_counter()

b = make_counter()

print(a())

print(a())

print(b())

print(a())

A)1

2

1

3

B)1

2

3

4

C)1

1

1

1

D)0

1

0

2

Q11. What is the output?

```
try:  
    print(10 / 0)  
except Exception:  
    print("A")  
except ZeroDivisionError:  
    print("B")
```

A)A B

B)SyntaxError

C)A



D)B

Q12. Suppose module_a.py contains:

```
print("Module loaded")
```

```
value = 10
```

And another file contains:

```
import module_a  
import module_a
```

```
print(module_a.value)
```

Assuming both imports occur in the same Python process, what is normally printed?

A) ImportError

B) 10

10

C) Module loaded

Module loaded

10

D) Module loaded

10

Q13. What happens when this code executes?

```
def fun():  
    print(variable)  
    variable = 50
```

```
variable = 10
```

```
fun()
```

A) 10

B) UnboundLocalError

C) 50

D) NameError

Q14. Suppose sample.py contains:

```
x = 10
```

```
def display():  
    print("Hello")
```

What will this print?

```
import sample
```

```
print(sample.x)  
sample.display()
```

A) Only Hello

B) Error

C) x

display

D) 10

Hello

Q15. What happens here?

```
try:
```

```
    raise ValueError("Wrong value")
```

```
except ValueError as e:
```

```
    print(e)
```

A) Nothing

B) Wrong Value

C) e

D) ValueError

Q16. What will be the output?

```
import math
```

```
print(math.sqrt(25))
```

A) 5

B) 5.0

C) sqrt(25)

D) 25

Q17. What is the major difference between the following?

```
import math
```

```
and
```

```
from math import sqrt
```

A) There is no difference

B) First imports only sqrt

C) Both import only sqrt

D) First provides the module namespace; second imports sqrt directly

Q18. What is the output?

```
def fun(x):
```

```
    if x == 0:
```

```
        return
```

```
    return x + fun(x - 1)
```



print(fun(3))

- A)6
- B)3
- C)5
- D)None

Q19. What will be printed?

```
def fun(a, b):  
    return a + b
```

print(fun(10, b=20))

- A)SyntaxError
- B)30
- C)TypeError
- D)20

Q20. What is the output?

```
try:  
    raise ValueError  
except ValueError:  
    print("A")  
    raise
```

- A)A, and execution ends normally
- B)A, followed by propagation of the same exception
- C)No Output
- D)B

Q21. What will be printed?

x = 10

f = lambda: x

x = 20

print(f())

- A)NameError
- B)20
- C)10
- D)None

**Q22. if test.py has
print("A")**

```
if __name__ == "__main__":  
    print("B")
```

**If test.py is executed directly, what
will be printed?**

- A)B

B)Nothing

C)A followed by B

D)A

Q23. What will normally happen?

import math

import math

- A)The module is normally loaded and executed twice
- B)SyntaxError
- C)The module is normally loaded once and reused
- D)ImportError

Q24. What will be printed?

```
try:  
    x = int("10")  
except ValueError:  
    print("A")  
else:  
    print("B")  
finally:  
    print("C")
```

- A)B
- B)A
- C)B C
- D)A C

Q25. What is the output?

```
def fun():  
    try:  
        return 10  
    finally:  
        return 20
```

print(fun())

- A)RuntimeError
- B)20
- C)None
- D)10

Q26. What happens here?

```
class MyError(Exception):  
    pass
```

```
def test():  
    raise MyError("Something went  
wrong")
```



```
try:
    test()
except MyError as e:
    print(type(e).__name__)
```

- A)TypeError
- B)Something went wrong
- C)Exception
- D)MyError

Q27. What is the output?

```
def fun(a=10, b=20):
    return a - b
```

```
print(fun(30))
```

- A)30
- B)20
- C)-10
- D)10

Q28. What is the output?

```
f = lambda x: x + 5
```

```
print(f(10))
```

- A)10
- B>Error
- C)5
- D)15

Q29. Suppose the current directory contains a file named random.py. What potential problem can occur with:

```
import random
```

- A)The local file can shadow the intended standard-library module
- B)random cannot be imported
- C)Python will always reject it
- D)Python automatically renames the local file

Q30. What will be printed?

```
def fun():
    try:
        return 10
    finally:
        print("Finally")
```

```
print(fun())
```

- A)10 followed by Finally
 - B)Finally followed by 10
 - C)10 only
 - D)Finally only
- Q31. What will be printed?**

```
def fun(x=[]):
    x.append(10)
    return x
```

```
a = fun()
b = fun()
```

```
print(a)
```

```
print(b)
```

- A)[10]
- [10]
- B)[10, 10]
- [10]
- C)[10]
- [10, 10]
- D>Error

Q32. What is the output?

```
def fun():
    try:
        print("A")
        raise ValueError
    except ValueError:
        print("B")
        return 10
    else:
        print("C")
        return 20
    finally:
        print("D")
```

```
print(fun())
```

- A)A
- B
- 10
- D
- B)A
- C
- D
- 20



C)A
B
C
D
10
D)A
B
D
10

Q33. What will be printed?

```
from functools import reduce
```

```
numbers = [1, 2, 3, 4]
```

```
result = reduce(  
    lambda a, b: a * b,  
    numbers  
)
```

```
print(result)
```

A)0
B)24
C)9
D)10

Q34. What is the output?

```
def outer():  
    x = 10
```

```
    def inner():  
        nonlocal x  
        x = x + 5  
        return x
```

```
    inner()  
    return x
```

```
print(outer())
```

A)UnboundLocalError
B)15
C)10
D)5

Q35. What is the purpose of this statement?

```
if __name__ == "__main__":
```

A)It identifies code intended to run when the file is executed directly
B)It prevents all imports
C)It creates the main function
D)It prevents syntax errors

Q36. What is the output?

```
numbers = [1, 2, 3, 4, 5]
```

```
result = list(  
    map(  
        lambda x: x * 2,  
        filter(lambda x: x % 2 == 0,  
            numbers)  
    )  
)
```

```
print(result)
```

A)[4, 8]
B)[1, 3, 5]
C)[2, 4]
D)[2, 4, 6, 8, 10]

Q37. Suppose test.py contains:

```
print("A")
```

```
if __name__ == "__main__":  
    print("B")
```

If another Python program executes:

```
import test
```

what will normally be printed?

A)A and B
B)B
C)Nothing
D)A

Q38. What happens here?

```
from math import sqrt
```

```
print(sqrt(36))
```

A)6.0
B)36
C)6
D)NameError



Q39. What is the output?

```
def fun(x):  
    return x * 2
```

```
a = fun  
print(a(5))
```

- A)5
- B)Error
- C)10
- D)func(5)

Q40. What will be the output?

```
x = 10
```

```
def fun():  
    print(x)  
    x = 20
```

```
fun()
```

- A)NameError
- B)20
- C)UnboundLocalError
- D)10

Q41. What will be the output?

```
x = 10
```

```
def outer():  
    x = 20
```

```
    def inner():  
        global x  
        x = 30
```

```
    inner()  
    print(x)
```

```
outer()  
print(x)
```

- A)30
- 30
- B)30
- 20
- C)20
- 30
- D)20
- 10

Q42. What will be printed?

```
def fun():  
    print("A")  
    return  
    print("B")
```

```
fun()
```

- A)B
- B)A B
- C)A
- D)Nothing

Q43. What will be the output?

```
def fun(x):  
    try:  
        if x < 0:  
            raise ValueError("negative")  
        return x  
    except ValueError:  
        return 100  
    finally:  
        print("cleanup")
```

```
print(fun(-5))
```

- A)100
- cleanup
- B)cleanup
- 100
- C)cleanup
- 5
- D)negative
- cleanup

Q44. What happens when this function is called?

```
def fun(a=10, b):  
    return a + b
```

- A)TypeError
- B)NameError
- C)It executes correctly
- D)SyntaxError

Q45. Which object keeps track of modules that have already been imported during the current Python process?

- A)sys.path
- B)sys.imports



C) __module__

D) sys.modules

Q46. What will be printed?

```
x = 10
```

```
def outer():
```

```
    x = 20
```

```
    def inner():
```

```
        print(x)
```

```
    x = 30
```

```
    inner()
```

```
outer()
```

A) NameError

B) 10

C) 30

D) 20

Q47. What will happen?

```
def outer():
```

```
    x = 10
```

```
    def inner():
```

```
        nonlocal y
```

```
        y = 20
```

```
    inner()
```

```
outer()
```

A) NameError

B) 10

C) SyntaxError

D) 20

Q48. What is the output?

```
def fun(x):
```

```
    x += 10
```

```
    return x
```

```
x = 5
```

```
y = fun(x)
```

```
print(x, y)
```

A) 5 5

B) 5 15

C) 15 5

D) 15 15

Q49. What happens?

```
def fun(a, b):
```

```
    return a + b
```

```
print(fun(10, a=20))
```

A) TypeError

B) SyntaxError

C) 20

D) 30

Q50. Why is

try:

```
    print(10 / 0)
```

except Exception:

```
    print("A")
```

except ZeroDivisionError:

```
    print("B")
```

codes's second handler never reached?

A) ZeroDivisionError occurs before Exception

B) except blocks execute randomly

C) Exception is a superclass of ZeroDivisionError

D) ZeroDivisionError is not an exception

